BSCS FINAL PROJECT

Software Requirements Specification – Version 1

AI-Powered Interview Preparation Assistant

Project Advisor

Arsalan Tariq

Presented by:

Warda Kaleem
Class IT
7th semester
Batch 2023
Roll number BS-IT-23-25

Department of Computer Science

Government College University Faisalabad

Government College University Faisalabad

Software Requirements Specification

Version 1.0

AI-Powered Interview Preparation Assistant

Client: Not Applicable – Academic Project

Team : Individual Project

Member Name	Primary Responsibility
Warda Kaleem	Requirements analysis, project documentation, GUI design, NLP/LLM integration, implementation, and testing

Table of Contents

Table of Contents ..... **ii**
Revision History ..... **iii**
1. Introduction **1**
1.1 Product 1
1.2 Scope 1
1.3 Business Goals 1
1.4 Document Conventions 1
1.5 References 1
2. Overall Description **2**
2.1 Product Features 2
2.2 User Classes and Characteristics 2
2.3 Operating Environment 2
2.4 Design and Implementation Constraints 2
2.5 Assumptions and Dependencies 2
3. Functional Requirements **3**
3.1 Use-Case 1 3
3.2 Use-Case 2 (and so on) 3
4. Nonfunctional Requirements **3**
4.1 Performance Requirements 3
4.2 Safety Requirements 4
4.3 Security Requirements 4
4.4 Software Quality Attributes 4
5. Other Requirements **4**
Appendix A: Glossary **5**
Appendix B: Analysis Models **6**
Appendix C: Design Models **7**
Appendix D: Screenshots **8**
Appendix E: Test Cases **9**
Appendix F: IV & V Report 10

Revision History

Name	Date	Reason For Changes	Version
Warda Kaleem	15-09-2026	Initial Software Requirements Specification prepared for the proposed project	V1.0

1. Introduction

The AI-Powered Interview Preparation Assistant is a proposed desktop-based application designed to help students and job seekers practice for technical and behavioral interviews. The application will use Natural Language Processing and Large Language Model techniques to generate interview questions for a selected job role, accept answers typed by the user, evaluate those answers, and provide feedback and a score.

The project is intentionally limited to a simple graphical user interface and text-based interaction. It will not function as an actual recruitment or hiring system. Its purpose is to provide an educational and self-practice tool that helps users improve the relevance, clarity, and completeness of their interview answers.

1.1 Product

The product will be a lightweight desktop application with a graphical user interface. The user will select an interview category or job role, such as Software Engineering or General Behavioral/HR Interview. The application will then display or generate relevant interview questions.

The user will type an answer to each question. The application will analyze the answer using NLP/LLM-based techniques and display feedback related to answer relevance, completeness, clarity, and possible areas of improvement. At the end of the session, the application will display a basic performance summary and score.

The first version of the product will focus on text-based question answering. Voice input, video analysis, facial-expression analysis, online deployment, user accounts, and automated recruitment decisions are not included in the initial version.

1.2 Scope

The scope of this project includes the analysis, design, and development of a simple desktop GUI application for interview practice.

The application will include the following main functions:

- Allow the user to select an interview category or job role.
- Display or generate relevant interview questions.
- Allow the user to enter an answer using a text field.
- Evaluate the answer using NLP/LLM-based techniques.
- Display a score and short feedback.
- Display a summary at the end of the practice session.
- The application will initially support a limited number of categories, such as Software Engineering and General Behavioral/HR interviews.

The following items are outside the scope of the initial version:

- Voice-based answer recording.
- Facial-expression or body-language analysis.
- Online recruitment or candidate selection.
- Automatic hiring decisions.
- Multi-user online access.
- Mobile application development.
- Web-based deployment.
- Complex user authentication.
- Training a large language model from scratch.
- The system will use pretrained NLP/LLM models or model APIs where appropriate. Sentence-transformer models may be used to compare the meaning of a user's answer with expected answer points. Sentence Transformers are commonly used for semantic textual similarity by converting text into embeddings and comparing their similarity. (sbert.net)

1.3 Business Goals

Although this is an academic project, the proposed system addresses the following practical goals:

- Provide students with an affordable interview practice tool.
- Provide instant feedback without requiring a human interviewer for every practice session.
- Help users identify weaknesses in the relevance, clarity, and completeness of their answers.
- Support self-paced preparation for technical and behavioral interviews.
- Demonstrate the practical use of NLP and LLM technologies in an educational application.
- Provide a foundation that could later be extended with additional job roles, question categories, or voice-based interaction.

1.4 Document Conventions

The following conventions are used:

- Section headings are numbered according to the template.
- Functional requirements are described through use cases.
- The terms "shall" and "must" indicate mandatory requirements.
- The term "should" indicates a desirable requirement that may be implemented if time and resources permit.
- The term "may" indicates an optional feature.
- Technical terms and abbreviations are defined in Appendix A: Glossary.
- Version 1 describes the initial requirements and planned behavior of the system. It does not indicate that implementation or testing has been completed.
- Use-case identifiers are written in the format UC-1, UC-2, and so on.
- Requirement identifiers are written in the format FR-1, FR-2, and so on, where appropriate.

1.5 References

[1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pp. 3982–3992, 2019.
[2] Sentence Transformers Documentation, “Semantic Textual Similarity,” documentation for generating text embeddings and calculating similarity between texts. Accessed September 15, 2026. (sbert.net)
[3] Hugging Face, “Transformers Documentation: Pipelines,” documentation describing the use of pretrained models for tasks including text generation and natural language processing. Accessed September 15, 2026. (github.com)
[4] E. Tabassi, “Artificial Intelligence Risk Management Framework: AI RMF 1.0,” National Institute of Standards and Technology, NIST AI 100-1, January 2023. (nist.gov)
[5] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, pp. 2383–2392, 2016.

2. Overall Description

The AI-Powered Interview Preparation Assistant will be a standalone desktop application intended for students and job seekers. The system will provide a controlled practice environment in which users can answer interview questions and receive automated feedback.
The application will use a simple graphical interface. The user will select an interview category, answer questions one by one, submit each answer, and view the generated feedback. The system will not replace a professional interviewer and will not be used for actual employment decisions.

2.1 Product Features

The main features planned for the initial version of the product are:

1. **Interview Category Selection**
   The user can select a category such as Software Engineering or General Behavioral/HR Interview.
2. **Interview Question Display or Generation**
   The system can display questions from a curated question bank and may generate or vary questions using a pretrained LLM.
3. **Text-Based Answer Input**
   The user can type an answer using the application's text input area.
4. **Answer Evaluation**
   The system evaluates the answer for relevance, completeness, and clarity using NLP/LLM-based methods.
5. **Feedback Generation**
   The system provides short feedback describing the strengths and weaknesses of the answer.
6. **Answer Score**
   The system provides a basic score for each answer according to the selected evaluation criteria.
7. **Session Summary**
   At the end of the session, the system displays the number of questions attempted, individual scores, and an overall summary.
8. **Reset or New Session Option**
   The user can start a new practice session without restarting the complete application.

2.2 User Classes and Characteristics

The initial version of the system will have one main user class.

Student or Job Seeker

The student or job seeker is the primary user of the application. This user will:

- Select an interview category.
- Read interview questions.
- Type answers.
- Submit answers for evaluation.
- Read feedback and scores.
- Review the final session summary.

The user is expected to have basic computer literacy but does not require technical knowledge of NLP, LLMs, or machine learning. The application should therefore provide a simple and understandable graphical interface.
No separate administrator user is required in the initial version. The question bank and configuration files may be maintained by the developer during the project.

2.3 Operating Environment

The application is planned to operate as a desktop application on a personal computer or laptop.

Expected software environment:

- Operating system: Windows 10 or Windows 11.
- Programming language: Python.
- GUI framework: Tkinter or PyQt.
- NLP/ML libraries: Sentence Transformers, Hugging Face Transformers, scikit-learn, and other required Python libraries.
- Storage: Local files such as JSON/CSV files or a lightweight SQLite database, if required.
- Internet connection: May be required if an external LLM API is used. If a local pretrained model is selected, the core application may operate without continuous internet access.
-

Expected hardware environment:

- A standard personal computer or laptop.
- Minimum 4 GB RAM; 8 GB or more is preferred.
- Sufficient storage for the Python environment, datasets, and selected pretrained models.
- Keyboard and display for text-based interaction.
-

The exact hardware and model requirements will be confirmed during implementation.

2.4 Design and Implementation Constraints

The following constraints will affect the design and implementation of the system:

1. The project must be completed within the university's final-year project schedule.
2. The application must remain simple enough to be implemented and tested by a student project team.
3. The initial version will use a desktop GUI rather than a web application.
4. The system will use pretrained NLP/LLM models or APIs instead of training a large language model from scratch.
5. The available computing resources may limit the use of large local models.
6. The quality and consistency of generated questions and feedback may depend on the selected model.
7. The application must not make actual hiring or employment decisions.
8. The system should avoid storing unnecessary personal information.
9. The application should provide understandable feedback rather than presenting its score as a guaranteed measure of interview success.
10. The initial version will focus on English text input.

2.5 Assumptions and Dependencies

The project is based on the following assumptions and dependencies:

1. Users will provide readable and meaningful text answers.
2. The selected NLP or LLM model will be available through a local Python library or an external API.
3. If an external API is used, an internet connection and valid API access will be required.
4. Publicly available interview-question material and a manually curated question bank will be available for the initial categories.
5. The system will initially focus on a limited number of categories rather than supporting every possible profession.
6. The feedback produced by the system will be considered guidance and not an official assessment.
7. A pretrained sentence-embedding model may be used to compare the semantic relationship between a user's answer and reference answer points.
8. The final choice between an external LLM API and a local model will be made after evaluating available resources, cost, response time, and ease of implementation.
9. The project will depend on Python libraries such as Tkinter or PyQt, Sentence Transformers, and Hugging Face Transformers.

Any change in model availability, API access, computing resources, or project scope may require revision of these requirements.

3. Functional Requirements

3.1 Select Interview Category

Identifier: UC-1
Purpose: Allow the user to choose the type of interview practice session.
Priority: High
Pre-conditions:
- The application is running.
- The category list is available.

Post-conditions:
- The selected category is stored for the current session.
- The system is ready to display or generate questions for that category.

Typical Course of Action

S#	Actor Action	System Response
1	User opens the application.	The system displays the main screen.
2	User views the available categories.	The system displays categories such as Software Engineering and General Behavioral/HR.
3	User selects a category.	The system records the selected category.
4	User clicks the Start Session button.	The system starts a new interview practice session.

3.1.1 Alternate Course of Action

S#	Actor Action	System Response
1	User attempts to start without selecting a category.	The system displays a message asking the user to select a category.
2	User selects an unavailable or invalid category.	The system displays an error message and asks the user to select another category.

3.2 Display or Generate Interview Question

Identifier: UC-2
Purpose: Provide an interview question relevant to the selected category.
Priority: High
Pre-conditions:
- The user has selected an interview category.
- A question bank or question-generation component is available.

Post-conditions:
- A question is displayed to the user.

- The user can enter an answer.

3.2.1 Typical Course of Action

S#	Actor Action	System Response
1	User starts an interview session.	The system loads the selected category.
2	The system selects or generates a question.	The system displays the question in the GUI.
3	User reads the question.	The system keeps the question visible while the user prepares an answer.
4	User clicks the answer area.	The system allows the user to enter a text response.

3.2.2 Alternate Course of Action

S#	Actor Action	System Response
1	The question-generation service is unavailable.	The system displays a question from the local question bank, if available.
2	No question is available for the selected category.	The system displays an appropriate error message.

3.3 Submit Interview Answer

Identifier: UC-3
Purpose: Allow the user to submit a written answer for evaluation.
Priority: High
Pre-conditions:

- A question is displayed.
- The answer input area is available.

Post-conditions:

- The answer is submitted for evaluation.
- The answer is associated with the current question.

Typical Course of Action

S#	Actor Action	System Response
1	User types an answer in the text area.	The system displays the entered text.
2	User clicks the Submit Answer button.	The system checks whether an answer has been entered.
3	The answer is valid.	The system sends the answer to the evaluation component.
4	Evaluation is completed.	The system displays the result and feedback.

3.3.1 Alternate Course of Action

S#	Actor Action	System Response
1	User submits an empty answer.	The system asks the user to enter an answer.
2	User enters an answer that is too short.	The system displays a warning and allows the user to revise or submit the answer.
3	Evaluation fails.	The system displays an error message and allows the user to try again.

3.4 Evaluate Answer and Generate Feedback

Identifier: UC-4
Purpose: Evaluate the user's answer and provide feedback.
Priority: High
Pre-conditions:

- A question and answer are available.
- The selected evaluation model or method is available.

Post-conditions:

- The system produces an evaluation result.
- Feedback and a score are displayed to the user.

Typical Course of Action

S#	Actor Action	System Response
1	User submits an answer.	The system receives the answer and the related question.
2	The system processes the text.	The system analyzes the answer using the selected NLP/LLM method.
3	The system compares the answer with evaluation criteria or reference points.	The system calculates a basic score.
4	Evaluation is completed.	The system displays feedback about relevance, completeness, clarity, and possible improvement.
5	User reads the feedback.	The system allows the user to continue to the next question.

3.4.1 Alternate Course of Action

S#	Actor Action	System Response
1	The NLP/LLM component is unavailable.	The system displays an error and, where possible, provides a basic local evaluation.
2	The answer does not contain enough information.	The system provides feedback that the answer should be expanded or made more specific.
3	The system cannot confidently evaluate the answer.	The system informs the user that the result is an approximate practice assessment.

3.5 3.5 View Session Summary

Identifier: UC-5
Purpose: Display the results of the completed practice session.
Priority: Medium
Pre-conditions:
- The user has answered at least one question.
- The application has recorded the evaluation results.

Post-conditions:
- The system displays the session summary.
- The user can start a new session or close the application.

Typical Course of Action

S#	Actor Action	System Response
1	User completes the final question.	The system records the final evaluation.
2	User clicks the Finish Session button.	The system calculates the session summary.
3	The system completes the calculation.	The system displays the number of questions, individual scores, and overall score.
4	User reviews the results.	The system provides an option to start a new session or exit.

3.5.1 Alternate Course of Action

S#	Actor Action	System Response
1	User attempts to finish without answering any question.	The system displays a message that no results are available.
2	User closes the application before finishing.	The system exits without displaying a final summary.

4. Nonfunctional Requirements

4.1 Performance Requirements

1. The application should open the main GUI within 10 seconds on the target computer.
2. The application should display a locally stored question within 2 seconds after the user requests it.
3. When using a local evaluation model, the system should attempt to display feedback within 15 seconds for a normal-length answer.
4. When using an external LLM API, response time will depend on internet connection and service availability.
5. The application should remain responsive while processing an answer. If processing takes longer than expected, it should display a progress message.

6. The application should support answers of at least 1,000 words, although normal answers are expected to be shorter.
7. The application should not generate duplicate questions within the same practice session unless the available question bank is too small.

4.2 Safety Requirements

1. The application shall not be used to make employment, hiring, rejection, or promotion decisions.
2. The application shall clearly communicate that its scores and feedback are approximate practice guidance.
3. The application shall not present its evaluation as a professional psychological, academic, or employment assessment.
4. The system should not encourage users to provide highly sensitive personal information in their answers.
5. If an answer contains harmful, offensive, or inappropriate content, the application should display feedback in a neutral and non-discriminatory manner.
6. The application should fail safely if the model or external service is unavailable by displaying an understandable error message rather than producing misleading results.

4.3 Security Requirements

1. The initial version shall not require users to create an online account.
2. The application should store only the information required for the current practice session.
3. User answers should not be shared with third parties unless an external LLM API is deliberately used for evaluation and the user is informed.
4. If an external API is used, API credentials shall not be hard-coded or exposed in the application interface.
5. The application should not collect unnecessary personal information such as national identification numbers, home addresses, passwords, or financial information.
6. Temporary session data should be removed when the user closes the application, unless a future version explicitly adds local history storage.
7. The system should provide an error message without exposing API keys, file paths, or internal implementation details.
8. The system should be designed as a practice tool and should not store sensitive employment records.

4.4 Software Quality Attributes

- **Usability:** A first-time user should be able to start a practice session without formal training.
- **Simplicity:** The GUI should contain only the controls necessary to select a category, view a question, enter an answer, submit it, and view feedback.
- **Reliability:** The application should display an appropriate message instead of crashing when an invalid answer or unavailable model is encountered.
- **Maintainability:** Question categories, questions, and evaluation settings should be organized so that they can be updated without rewriting the complete application.
- **Portability:** The application should run on the selected Windows operating-system environment with the required Python dependencies installed.
- **Testability:** Each main feature should be testable independently using predefined test cases.
- **Understandability:** Feedback should be written in clear language appropriate for students and job seekers.
- **Transparency:** The system should explain that its score is an approximate practice score and not a guaranteed prediction of interview performance.

- **Extensibility:** Additional interview categories may be added in a future version.

5. Other Requirements

1. The application should maintain a local question bank in a manageable format such as JSON, CSV, or SQLite.
2. Each question should be associated with an interview category.
3. The evaluation component may use reference points, keywords, semantic similarity, or an LLM-based rubric.
4. The system should provide feedback using understandable labels such as:
   i. Relevance
   ii. Completeness
   iii. Clarity
   iv. Suggested improvement
   v. The system should use English as the initial language.
   vi. The application should provide buttons or controls for:
   vii. Start Session
   viii. Submit Answer
   ix. Next Question
   x. Finish Session
   xi. New Session
   xii. Exit
5. The system may use either a local pretrained model or an external LLM API. The final choice will be documented in a later version of the requirements and design documents.
6. No online payment, user subscription, or commercial recruitment integration is required.
7. The final project will include documentation explaining the selected NLP/LLM approach and its limitations.

Appendix A: Glossary

AI: Artificial Intelligence; computer systems designed to perform tasks that normally require human intelligence.
API: Application Programming Interface; a mechanism that allows one software system to communicate with another.
GUI: Graphical User Interface; the visual interface through which the user interacts with the application.
LLM: Large Language Model; a machine-learning model designed to process and generate human language.
NLP: Natural Language Processing; a field of AI concerned with the processing and analysis of human language.
Pretrained Model: A model that has already been trained on a large dataset and can be used or adapted for a specific application.
Question Bank: A structured collection of interview questions organized by category or job role.
Semantic Similarity: The degree to which two pieces of text have similar meaning. Sentence-transformer models can generate embeddings that are used to calculate this similarity. (sbert.net)
Text Embedding: A numerical representation of text that captures some of its meaning.
Use Case: A description of an interaction between an actor and a system to achieve a particular goal.
Behavioral Interview: An interview that evaluates a candidate's past behavior, communication, teamwork, and problem-solving experiences.
Technical Interview: An interview that evaluates technical knowledge and problem-solving ability related to a job role.
Reference Answer Points: Important concepts or elements expected in an answer and used as guidance during evaluation.

Appendix B: Analysis Models

To be completed during the analysis and functional specification phase

Appendix C: Design Models

To be completed during the analysis and functional specification phase

Appendix D: Screenshots

To be completed during the analysis and functional specification phase

Appendix E: Test Cases

To be completed during the analysis and functional specification phase

Identifier	TC-1
Priority	<Choose one from {High, Medium, Low}>
Related requirements(s)	<Include use-case identifier(s) for functional requirement(s) and SRS section/sub-section number(s) for other requirement(s).>
Short description	…
Pre-condition(s)	…
Input data	…
Detailed steps	…
Expected result(s)	…
Post-condition(s)	…

Table 1: TC-1

Appendix F: IV & V Report

(Independent verification & validation)

IV & V Resource

Name Signature

S#	Defect Description	Origin Stage	Status	Fix Time	
				Hours	Minutes
1					
2					
3					
…					

Table 2: List of non-trivial defects

CLASSIFICATION OF SRS

SRS VER. 1

(After 03 Weeks) Submission Date: 22-4-2014.

Revision History.....
1. Introduction.....
1.1 Product.....
1.2 Scope.....
1.3 Business Goals.....
1.4 Document Conventions.....
1.5 References.....
2. Overall Description.....
2.1 Product Features.....
2.2 User Classes and Characteristics.....
2.3 Operating Environment.....
2.4 Design and Implementation Constraints.....
2.5 Assumptions and Dependencies.....
3. Functional Requirements.....
3.1 Use-Case 1.....
3.2 Use-Case 2 (and so on).....
4. Nonfunctional Requirements.....
4.1 Performance Requirements.....
4.2 Safety Requirements.....
4.3 Security Requirements.....
4.4 Software Quality Attributes.....
5. Other Requirements.....
Appendix A: Glossary.....
Appendix F: IV & V Report……

SRS VER. 2

Submission Date: 03-06-2014.

Revision History.....
1. Introduction.....
1.1 Product.....
1.2 Scope.....
1.3 Business Goals.....
1.4 Document Conventions.....
1.5 References.....
2. Overall Description.....
2.1 Product Features.....
2.2 User Classes and Characteristics.....
2.3 Operating Environment.....
2.4 Design and Implementation Constraints.....
2.5 Assumptions and Dependencies.....
3. Functional Requirements.....
3.1 Use-Case 1.....
3.2 Use-Case 2 (and so on).....
4. Nonfunctional Requirements.....
4.1 Performance Requirements.....
4.2 Safety Requirements.....
4.3 Security Requirements.....
4.4 Software Quality Attributes.....
5. Other Requirements.....
Appendix A: Glossary.....
Appendix B: Analysis Models.....
Appendix C: Design Models.....
Appendix D: Screenshots.....
Appendix F: IV & V Report.....

Initial Operational Capabilities
(Core features implemented)

Final Documentation

Submission Date: 01-7-2014.

Revision History.....
1. Introduction.....
1.1 Product.....
1.2 Scope.....
1.3 Business Goals.....
1.4 Document Conventions.....
1.5 References.....
2. Overall Description.....
2.1 Product Features.....
2.2 User Classes and Characteristics.....
2.3 Operating Environment.....
2.4 Design and Implementation Constraints.....
2.5 Assumptions and Dependencies.....
3. Functional Requirements.....
3.1 Use-Case 1.....
3.2 Use-Case 2 (and so on).....
4. Nonfunctional Requirements.....
4.1 Performance Requirements.....
4.2 Safety Requirements.....
4.3 Security Requirements.....
4.4 Software Quality Attributes.....
5. Other Requirements.....
Appendix A: Glossary.....
Appendix B: Analysis Models.....
Appendix C: Design Models.....
Appendix D: Screenshots.....
Appendix E: Test Cases.....
Appendix F: IV & V Report……

(Complete Documentation with final working product)